

		Stream <code>.filter(...)</code> // reduce data <code>.distinct()</code> // remove duplicates <code>.map(...)</code> // transform <code>.flatMap(...)</code> // flatten if needed <code>.sorted(...)</code> // expensive → late <code>.peek(...)</code> // debug only <code>.collect(...);</code> // terminal operation
Operation	Use Case	One-Line Code Example
filter()	boolean checks	<code>.filter(x -> x > 10)</code>
filter()	null checks	<code>.filter(Objects::nonNull)</code>
filter()	comparisons	<code>.filter(e -> e.getSalary() >= 50000)</code>
filter()	method calls returning boolean	<code>.filter(User::isActive)</code>
-		
distinct()	rely on equals() / hashCode()	<code>.distinct()</code>
distinct()	immutable objects	<code>.map(String::toUpperCase).distinct()</code>
map()	transform objects	<code>.map(Employee::getName)</code>
map()	create new objects	<code>.map(e -> new EmployeeDTO(e.getId(), e.getName()))</code>
map()	call pure functions	<code>.map(Math::abs)</code>

flatMap()	return Stream	.flatMap(s -> Arrays.stream(s.split(", ")))
flatMap()	split data	.flatMap(sentence -> Arrays.stream(sentence.split(" ")))
flatMap()	expand collections	.flatMap(order -> order.getItems().stream())
-		
sorted()	natural ordering	.sorted()
sorted()	custom comparator	.sorted(Comparator.comparing(Employee::getSalary))
sorted()	multi-field compare	.sorted(Comparator.comparing(Employee::getDept).thenComparing(Employee::getSalary))
peek()	logging	.peek(e -> System.out.println(e))
peek()	debugging	.peek(e -> log.debug("Processing: {}", e))
peek()	metrics	.peek(e -> counter.incrementAndGet())
-		
collect()	accumulate data	.collect(Collectors.toList())
collect()	grouping	.collect(Collectors.groupingBy(Employee::getDepartment))
collect()	reduction	.collect(Collectors.summingInt(Employee::getSalary))
collect()	custom collectors	.collect(Collector.of(ArrayList::new, List::add, (a,b)->{a.addAll(b);return a;}))